Ministry of Higher Education and Scientific Research
M'hamed Bougara University of Boumerdès
Faculty of Sciences – Department of Computer Science

Practical Work / Tutorial 2	**Module: Advanced Databases**	**Academic Year: S1 – Master GL / I2A**	**ISHAK BOUSHAKI**

Topic: Views, SQL/PSM, Stored Functions, Procedures, Constraints and Triggers

Remark: Questions marked with an asterisk (*) are priority questions and should be completed first.

Database Schema

The following relational schema is used throughout the exercises:

- department(department_id, department_name)
- class(class_id, class_name, department_id)
- teacher(teacher_id, last_name, first_name, salary, department_id)
- student(student_id, last_name, first_name, age, class_id)

Assume that primary keys and foreign keys are appropriately defined. Some teachers may have a NULL salary, and some students may have a NULL class identifier.

Exercise 1 (*) — Standard Views: Creation, Nested Views, and Constraints

1. **Creating and Using a View:** Create a view named teacher_public_view displaying each teacher's identifier, last name, first name, and department identifier, without displaying the salary. Write a query to retrieve teachers whose first name is Ahmed.
2. **Creating a View Based on Multiple Tables:** Create a view named teacher_department_view displaying the teacher's identifier, last name, first name, salary, and department name by joining the teacher and department tables.
3. **Creating Nested Views:** Create a view named computer_science_teacher_view based on teacher_department_view, selecting only teachers belonging to the Informatique department. Then create another view named computer_science_high_salary_view based on the first view, displaying only teachers whose salary is greater than or equal to 70,000.
4. **Using WITH CHECK OPTION:** Create a view restricted to teachers belonging to the Informatique department using WITH CHECK OPTION. Explain the purpose of this clause and give an example of an update that should be rejected.
5. **Dropping a View:** Write the SQL statement to remove computer_science_high_salary_view.

Exercise 2 (*) — Materialized Views

1. Define a materialized view named department_average_salary that stores the average salary for each department.
2. Write a query to retrieve the stored average salary for each department.
3. Explain why the results of a materialized view may become outdated after a teacher's salary is modified.
4. Write the appropriate statement to refresh department_average_salary after a salary update.
5. Explain the difference between a regular view and a materialized view in terms of data storage, query execution, and result freshness.

6. Discuss one advantage and one limitation of using a materialized view to store department salary statistics.

Exercise 3 (*) — Iterating over Departments

Write a SQL/PSM code fragment that iterates over department identifiers from 1 to 5.

For each department, display its identifier.

Exercise 4 — Processing Students and Handling NULL Values

Consider the following requirements:

1. Process student identifiers from 1 to 10.
2. Retrieve the class identifier of each student.
3. Distinguish between a student whose class_id is NULL and a student who does not exist.
4. Explain why these two situations must not be treated as equivalent.

Exercise 5 — Recursive Common Table Expressions

1. Write a recursive common table expression (CTE) that generates the integers from 1 to 10.
2. Identify the anchor member and the recursive member.
3. Explain the termination condition.
4. Distinguish a recursive CTE from a procedural loop.

Exercise 6 — Handling Division by Zero

Write a SQL/PSM code fragment that performs a division and handles a division-by-zero error.

Explain the purpose of the exception handler and indicate which parts of the syntax may depend on the DBMS.

Exercise 7 (*) — Handling a Missing Student

Write a SQL/PSM code fragment that attempts to retrieve the information of student 100.

1. Explain how the program should behave if the student exists.
2. Explain how it should behave if the student does not exist.
3. Use an appropriate NOT FOUND or equivalent condition handler, according to the supported SQL/PSM implementation.
4. Explain why the handling of a missing row should be explicit.

Exercise 8 — Controlled Salary Update

Write a SQL/PSM procedure or code fragment that increases the salary of teacher 101 by 10%.

The solution must:

- Check that the teacher exists.
- Check that the salary is not NULL.
- Perform the update only if the checks succeed.
- Report an appropriate error or status if a condition is not satisfied.

Explain why validation should take place before the update.

Exercise 9 (*) — Counting Teachers by Department

Create a stored function named number_of_teachers(p_department) that returns the number of teachers belonging to the specified department.

Then write a query that displays each department's identifier, name, and number of teachers.

The expected counts are:

Department ID	Number of teachers
1	4
2	2
3	2
4	1
5	0

Explain why department 5 must appear in the result even though it has no teachers.

Exercise 10 — Average Salary by Department

Create a stored function named average_salary(p_department) that returns the average salary for a given department.

Test the function for departments 1 and 5.

Explain the result when a department has no teachers and why AVG may return NULL.

Exercise 11 — Salary Classification

Create a stored function named salary_category(p_teacher_id) that returns a salary category according to the following rules:

- High: salary greater than or equal to 70,000.
- Medium: salary greater than or equal to 50,000 and less than 70,000.
- Low: salary less than 50,000.
- Unknown: salary is NULL.

Explain how the function should handle a teacher identifier that does not exist.

Exercise 12 — Calculating a Salary Increase

Create a stored function named calculate_increased_salary(p_salary, p_rate) that calculates the salary after an increase.

Use the formula:

$$S_{\text{new}} = S_{\text{old}} \times \left(1 + \frac{r}{100}\right)$$

Specify how the function should handle a NULL salary and an invalid increase rate.

Exercise 13 — Increasing Salaries by Department

Create a stored procedure named increase_salaries(p_department, p_rate) that increases the salaries of all teachers belonging to a specified department.

The procedure must:

1. Check that the rate is valid.
2. Verify that the department exists.
3. Update the salaries of the teachers in that department.
4. Explain how teachers with a NULL salary should be handled.

Exercise 14 (*) — Safely Increasing One Teacher's Salary

Create a stored procedure named increase_teacher_salary(p_teacher_id, p_rate) that increases the salary of a single teacher.

The procedure must check that:

- The teacher exists.
- The salary is not NULL.
- The increase rate is positive.
- The update is performed only when all conditions are satisfied.

Exercise 15 — Conditional Salary Increases Using CASE

Write a procedure that applies a salary increase to every teacher according to the teacher's department:

Department	Increase rate
Informatique	5%
Mathématiques	7%
Électronique	6%
Physique	4%
Chimie	3%

Use a CASE expression to determine the rate.

Explain what should happen if a department has no teachers and how the procedure should handle a department that is not covered by the rules.

Exercise 16 (*) — Preventing Negative Salaries

Propose a solution to prevent a teacher's salary from becoming negative.

1. Explain how a CHECK constraint can enforce this rule.
2. Explain whether a trigger is necessary.
3. Discuss how the rule should treat a NULL salary.

Exercise 17 — Designing a Salary Audit Table

Design a table named salary_audit to record salary changes.

The table should contain:

- An audit identifier.
- The teacher identifier.

- The old salary.
- The new salary.
- The date and time of the change.

Explain the purpose of each column and identify the appropriate primary key.

Exercise 18 (*) — Creating a Salary Audit Trigger

Write a trigger that records salary changes in salary_audit.

The trigger must:

1. Execute after a salary update.
2. Record the teacher identifier, old salary, new salary, and change timestamp.
3. Avoid creating an audit record when the salary has not actually changed.
4. Explain how OLD and NEW values are used.